

LaserCtrl_FreeRTOS 项目总结与面试讲解

1. 项目背景

本项目最初是裸机版本，主逻辑集中在 `while(1)`：温度检测、串口接收、指令解析、DAC/I2C 输出、ADC 采样、日志输出都在同一循环内完成。

重构目标是升级为基于 FreeRTOS 的多任务架构，体现工程化设计能力、可维护性和可扩展性。

当前工程关键文件：

- `main.c`
 - `freertos.c`
 - `mcp4725.c`
 - `mcp4725.h`
 - `FreeRTOSConfig.h`
-

2. 系统目标与控制对象

系统控制 3 路激光器：

- Laser1：内部 DAC1 输出 + ADC1 CH0 反馈
- Laser2：内部 DAC2 输出 + ADC1 CH1 反馈
- Laser3：MCP4725(I2C) 输出 + ADC1 CH2/CH3 反馈

温度与优先级门控：

- Laser1：只依赖 Laser1 温度就绪
 - Laser2：依赖 Laser1 + Laser2 温度就绪
 - Laser3：依赖 Laser1 + Laser2 + Laser3 温度就绪（当前软件默认 Laser3 就绪）
-

3. 总体架构设计（FreeRTOS）

3.1 任务划分

在 `freertos.c` 中完成了任务拆分：

1. `UartRxTask`

- 职责：串口接收、过滤非法输入、把合法指令投递到命令队列。

2. `CommandTask`

- 职责：从命令队列取指令，执行优先级/温度校验，更新 DAC/I2C，触发 ADC 采样请求，计算并输出结果。

3. `TempMonitorTask`

- 职责：周期读取温度引脚，更新全局激光状态与使能 GPIO。

4. `AdcTask`

- 职责：等待采样请求信号量，执行 ADC DMA 采样，更新共享 ADC 数据。

5. `UartTxTask`

- 职责：统一串口日志发送，避免多任务同时抢占串口。

6. `defaultTask`

- 职责：系统安全初始化、设置 ready 事件标志、输出心跳日志。

3.2 RTOS 对象

1. 队列

- `uartCmdQueue`: `UartRxTask` -> `CommandTask`
- `uartTxQueue`: 其他任务 -> `UartTxTask`

2. 互斥锁

- `laserStateMutex`: 保护激光状态结构体
- `adcDataMutex`: 保护 ADC 采样结果
- `i2cMutex`: 保护 I2C 总线访问 (MCP4725)

3. 信号量

- `adcReqSem`: 命令任务发起采样请求
- `adcDoneSem`: ADC 任务完成采样后通知命令任务
- `adcCpltSem`: DMA 完成回调通知 ADC 任务

4. 事件标志

- `appFlags`: 系统初始化完成后发布 `APP_FLAG_READY`，各任务启动逻辑统一门控。

4. 关键实现思路

4.1 安全启动（上电默认安全）

在 `defaultTask` 初始化阶段：

- DAC1/DAC2 置 0
- MCP4725 置 0
- 按当前温度状态刷新三路使能 GPIO
- 再放开 `APP_FLAG_READY`

这样可保证激光输出在系统就绪前不会误开。

4.2 串口处理解耦

输入输出解耦：

- 输入由 `UartRxTask` 负责
- 输出由 `UartTxTask` 负责
- 中间用队列隔离

优点:

- 避免多处 `HAL_UART_Transmit` 抢占
- 日志顺序稳定
- 便于后续切换为 DMA/UART 中断发送

4.3 命令处理与业务规则

`CommandTask` 负责业务闭环:

1. 收指令
2. 门控校验 (温度/优先级)
3. 更新 DAC/I2C
4. 等待输出稳定 (Laser3 200ms)
5. 触发 ADC 采样并等待完成
6. 计算电压/电流并输出日志

步进与上限:

- Laser1/2 大步 31, 小步 3, 上限 3103
- Laser3 步进 20, 上限 1861

4.4 ADC 任务化

ADC 被设计为“请求-执行-返回”的后台服务:

- 命令任务发请求 (`adcReqSem`)
- ADC 任务执行 DMA 采样
- DMA 回调只做 `adcCpltSem` 释放
- ADC 任务写共享区并回通知 (`adcDoneSem`)

这符合 RTOS 最佳实践: 中断轻量化, 复杂逻辑留在任务上下文。

5. 从裸机到 RTOS 的核心收益 (面试重点)

5.1 可维护性

- 裸机: 逻辑集中在一个大循环, 耦合高。
- RTOS: 按职责拆任务, 边界清晰, 易读易改。

5.2 实时性与响应性

- 裸机: 某段逻辑阻塞会拖慢全局。
- RTOS: 通过优先级和阻塞等待实现更可控的调度。

5.3 并发安全

- 裸机：共享变量容易被随手访问。
- RTOS：共享资源通过 Mutex/Semaphore 受控访问。

5.4 可扩展性

- 后续可新增：
 - 故障管理任务 (Fault Manager)
 - 通讯协议任务 (如 Modbus/CAN)
 - 数据上报任务 (Telemetry)
 - 参数存储任务 (Flash/EEPROM)

6. 我在重构中的工程化实践

1. 先做“最小可用闭环”
 - 先打通串口收发和三路控制，确保可验证。
2. 再升级为多任务
 - 在已稳定基础上拆任务，避免一次重构风险过大。
3. 强化可观测性
 - 统一日志通道，关键状态可追踪（启动、温度、命令、异常）。
4. 降低中断复杂度
 - 中断只发信号，计算/打印全部放任务。
5. 面向面试表达
 - 明确“为什么这么拆、有什么收益、如何验证正确性”。

7. 你在面试可直接讲的“30秒版本”

“我把一个 STM32 裸机激光控制项目重构成了 FreeRTOS 多任务架构。

我按职责拆成串口接收、命令处理、温度监控、ADC采样、串口发送五个核心任务，任务间用队列和信号量通信，用互斥锁保护 I2C、激光状态和 ADC 数据。

这样把原来 `while(1)` 的耦合逻辑拆开了，系统更稳定、可维护，也更容易扩展到故障管理和远程通信。

我还实现了安全启动策略：上电先把 DAC 和激光使能全部置安全态，再释放系统 ready 标志，避免误触发高风险外设。”

8. 你在面试可讲的“深挖点”

1. 为什么 UART TX 要单独任务化？
 - 避免并发打印导致日志乱序和串口阻塞扩散。
2. 为什么 ADC 用任务 + 信号量，而不是全在中断里做？

- 中断应短小；复杂计算放任务更安全、可维护。
3. 如何避免系统上电误开激光？
- 先输出全置零，再根据温度和优先级统一下发使能。
4. 为什么需要事件标志 `APP_FLAG_READY`？
- 统一任务启动门控，避免任务抢跑访问未初始化资源。
5. 你如何验证重构没有破坏业务规则？
- 用串口指令 `0~9` 回归测试三路输出变化、门控拒绝、ADC反馈逻辑。
-

9. 后续可继续优化（加分项）

1. UART RX 改为中断或 DMA 环形缓冲
 2. 引入 watchdog + fault state machine
 3. 将日志分级（INFO/WARN/ERROR）
 4. 增加 CLI 命令（查询状态、在线调参）
 5. 对关键任务做栈水位监控与运行时统计
-

10. 总结

这次重构不是“把裸机代码搬到 RTOS”，而是完成了工程化架构升级：

- 从单循环到多任务协作
- 从共享变量随意访问到并发受控
- 从功能可跑到可维护、可讲解、可扩展

对于大厂面试，这类重构最能体现：

- 系统设计能力
- 并发与实时系统理解
- 工程质量意识（安全、可观测、可扩展）